
Unidades de Procesamiento Gráfico

Los sistemas de procesamiento gráfico (o GPUs, por sus siglas en inglés), son otra arquitectura de procesamiento que aprovecha el paralelismo masivo, creando desde centenas hasta miles de unidades funcionales capaces de ejecutar instrucciones. En el contexto de NVIDIA, generalmente se manejan dos términos: CUDA cores, que son unidades funcionales capaces de ejecutar aritmética entera, lógica, cargas/descargas a registros e instrucciones de punto flotante. Luego, cada CUDA core se agrupa en grupos de 32 para formar un procesador de stream (*stream multiprocessor*). La nomenclatura varía de fabricante en fabricante, pero la filosofía es similar.

Desde la perspectiva de programación, el paradigma que se utiliza es el de *Single-Instruction Multi-threads* (SIMT), donde una tarea ejecuta múltiples hilos homogéneos, que paralelizan el trabajo con el mismo set de instrucciones. Luego, estos hilos llegan a las unidades de ejecución y el *hardware* ejecuta una misma instrucción común en todos los hilos.

En este curso, estaremos abordando la GPU desde la perspectiva de hardware, luego la de software y como podemos utilizarlas para acelerar tareas de procesamiento, incluyendo las consideraciones de cuándo son competentes y cuándo no.

3.1. Fundamentos de organización

Primero, hay que entender cómo está organizada una GPU internamente. Puede asociarse como un procesador vectorial, donde cada unidad funcional es una línea de ejecución escalar dentro del vector, donde el número de elementos es siempre 32, sin importar el tipo de datos. El procesador como un todo es lo análogo a los *stream multiprocessors* o *compute units*, de NVIDIA y AMD/Intel respectivamente.

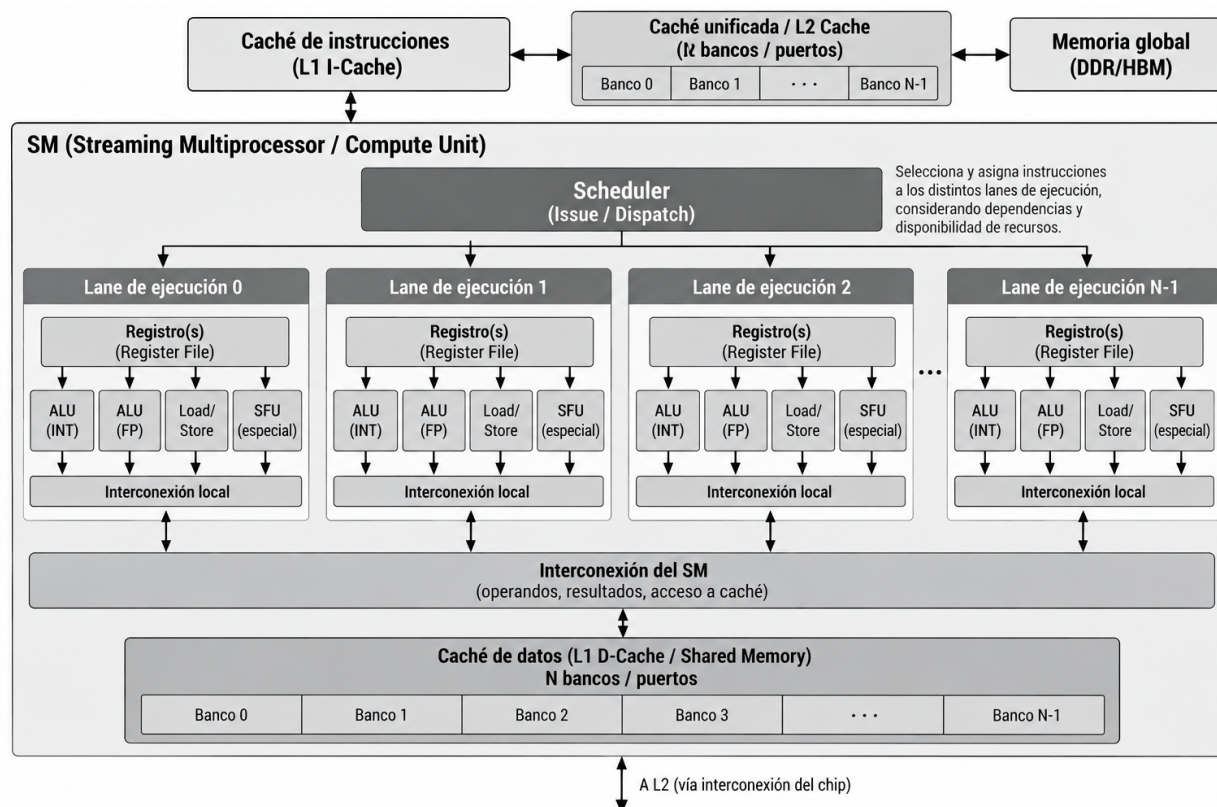


Figura 3.1: Organización de una GPU: sobre simplificada. Generada con ayuda de la IA y adaptada de [9]

Sistema de ejecución en arquitecturas GPU

En la Figura 3.1 se muestra una sobresimplificación de como luce la organización de una GPU. En esta figura se observa que un cluster de unidades funcionales consta de un *scheduler*, que permite la ejecución en paralelo de N instrucciones de la misma naturaleza. En este caso, cada unidad funcional se encuentra dentro de una línea de ejecución, que integra su propio banco de registros, ALU, unidad de L/D y una unidad especial, para el cálculo de instrucciones matemáticas complejas

de forma rápida, con cierto grado de aproximación. Todas las líneas de ejecución se intercomunican entre sí, con la finalidad de recolectar datos, resultados, u otros. En algunos casos, se cuenta con una memoria caché interna, donde cada línea puede tener acceso de forma inmediata, mediante un puerto dedicado.

Más concretamente, en NVIDIA se utilizan un total de 32 líneas de ejecución, lo que permite ejecutar 32 hilos homogéneos simultáneamente. La Figura 3.2 muestran la microarquitectura AMD RDNA 2 y la NVIDIA Ampere. Se puede observar que la idea arquitectónica sigue un patrón similar: existe un front-end que permite ordenar y despachar las instrucciones a ejecutar, mientras que existe un *engine* de ejecución que tiene acceso a registros vectoriales y luego las unidades funcionales se comunican mediante algún método de interconexión. La mayor diferencia entre AMD y NVIDIA en cuanto a la organización de la ejecución es la presencia de una mayor densidad de unidades funcionales en AMD, mientras que NVIDIA integra un motor de tensores, llamado *Tensor Cores*, que sirven para ejecutar multiplicación de matrices de forma más óptima.

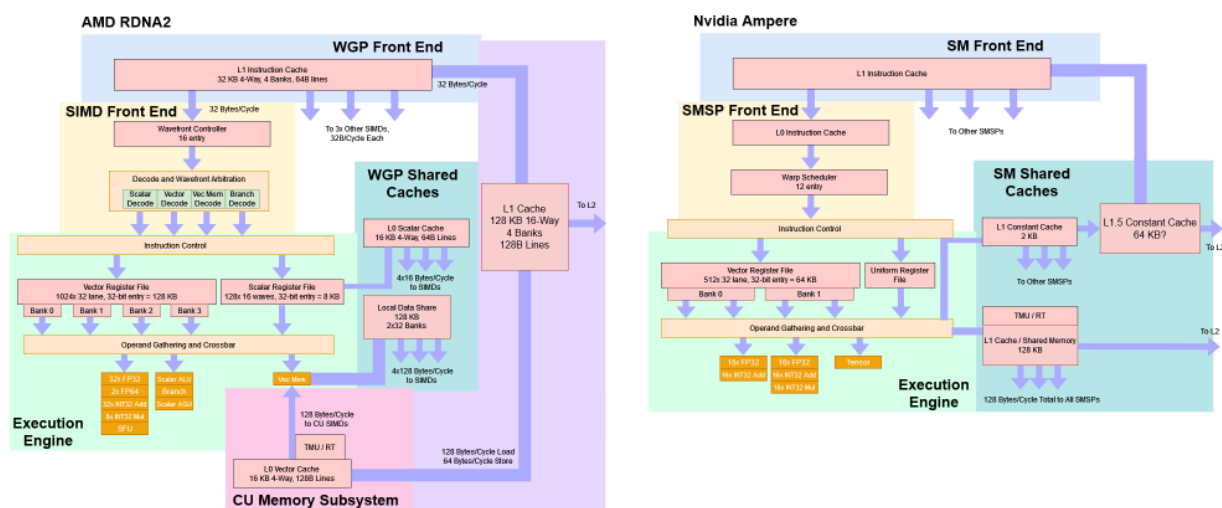


Figura 3.2: AMD RDNA and NVIDIA Ampere Microarchitectures. Obtenido de [9]

Sistema de memoria en arquitecturas GPU

Las GPU modernas requieren un sistema de memoria capaz de alimentar una gran cantidad de unidades de ejecución que operan simultáneamente. A diferencia de una CPU convencional, donde una parte importante de la jerarquía de memoria busca reducir la latencia observada por un número relativamente pequeño de núcleos, una GPU debe además proporcionar un elevado ancho de banda agregado para mantener ocupadas sus múltiples unidades de ejecución.

La Figura 3.2 muestra dos ejemplos de esta organización utilizando las arquitecturas AMD RDNA 2 y NVIDIA Ampere. Aunque la nomenclatura y organización interna difieren, ambas utilizan

registros, memorias locales, cachés y múltiples caminos de acceso a memoria para proporcionar datos e instrucciones a las unidades de ejecución.

Con respecto a la jerarquía de memoria a lo interno de RDNA 2, las unidades de cómputo se organizan alrededor del *Work Group Processor* (WGP). En la parte superior de la jerarquía mostrada se encuentra una caché de instrucciones L1 de 32 KiB, compartida entre los SIMD asociados al WGP. Esta memoria permite alimentar los *front-ends* encargados de seleccionar, decodificar y distribuir las instrucciones que serán ejecutadas por los diferentes *wavefronts*. RDNA 2 incorpora también memorias especializadas cercanas a las unidades de ejecución. Entre ellas se encuentra la *Local Data Share* (LDS), una memoria explícitamente compartida por los hilos de un grupo de trabajo. La organización mostrada dispone de 128 KiB distribuidos en 32 bancos. Esta organización en bancos permite realizar varios accesos simultáneos siempre que estos puedan distribuirse entre bancos diferentes. Adicionalmente, la arquitectura emplea cachés diferenciadas para distintos tipos de acceso. La figura muestra una caché vectorial L0 de 16 KiB y una caché escalar L0 de 16 KiB. Los accesos que no pueden resolverse en estos niveles continúan hacia una caché L1 compartida por el WGP, mostrada con una capacidad de 128 KiB y organizada en cuatro bancos. Finalmente, los accesos pueden propagarse hacia los niveles superiores de la jerarquía, como L2 y, posteriormente, la memoria global.

Por tanto, una representación simplificada de la jerarquía observada en RDNA 2 puede expresarse como:

Registros $\rightarrow$ L0 / LDS $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ Memoria global

NVIDIA Ampere utiliza una organización conceptualmente similar, aunque con una distribución diferente de sus recursos. Las unidades de ejecución se encuentran agrupadas dentro de un *Streaming Multiprocessor* (SM), el cual se subdivide internamente en varias particiones de procesamiento o SMSP. En el camino de instrucciones, el SM dispone de una caché de instrucciones L1, mientras que cada SMSP posee una caché de instrucciones L0 más cercana al *warp scheduler*. De esta forma, las instrucciones utilizadas por los *warps* pueden mantenerse cerca de las unidades responsables de su planificación y ejecución.

Al igual que en RDNA 2, una parte fundamental del sistema de memoria se encuentra en los registros. Cada partición dispone de un *Vector Register File* desde el cual se obtienen los operandos para las unidades funcionales. La arquitectura incluye diferentes caminos de ejecución para operaciones FP32, INT32 y operaciones matriciales realizadas por los *Tensor Cores*.

Para los datos, la figura muestra una estructura combinada denominada *L1 Cache / Shared Memory*, con una capacidad de 128 KiB. Esta organización es especialmente importante porque una parte de este recurso puede emplearse como memoria compartida explícitamente gestionada por el programador, mientras que otra parte participa en el almacenamiento en caché de los accesos a memoria.

De manera simplificada, la jerarquía puede representarse como:

Registros $\rightarrow$ L0 / Shared Memory / L1 $\rightarrow$ L2 $\rightarrow$ Memoria global

3.2. Modelo de ejecución SIMT

Las GPU modernas utilizan principalmente un modelo de ejecución denominado SIMT. Este modelo permite ejecutar una gran cantidad de hilos concurrentemente, agrupándolos para que una misma instrucción pueda ser ejecutada sobre múltiples datos.

Desde el punto de vista del programador, cada hilo posee su propio estado, incluyendo registros, identificador y flujo de ejecución. Sin embargo, el hardware agrupa varios hilos en una unidad de planificación. NVIDIA denomina a estos grupos *warps*, mientras que AMD utiliza el término *wavefronts*. El *scheduler* selecciona uno de estos grupos y emite una instrucción hacia múltiples *lanes* de ejecución:

Warp/Wavefront $\rightarrow$ Scheduler $\rightarrow$ Instrucción $\rightarrow$ N lanes de ejecución

Por ejemplo, considérese la operación

$$C[i] = A[i] + B[i]. \quad (3.1)$$

En un modelo SIMT pueden existir múltiples hilos ejecutando la misma instrucción, pero utilizando diferentes índices:

Hilo	Operación
0	$C[0] = A[0] + B[0]$
1	$C[1] = A[1] + B[1]$
2	$C[2] = A[2] + B[2]$
$\vdots$	$\vdots$
$N - 1$	$C[N - 1] = A[N - 1] + B[N - 1]$

Aunque conceptualmente existen N hilos independientes, cuando estos pertenecen al mismo *warp* o *wavefront*, el hardware puede emitir una misma instrucción hacia múltiples unidades de ejecución. Esta característica permite aprovechar eficientemente el paralelismo de datos.

La Figura 3.3 ilustra la correspondencia entre el modelo de programación y la organización del hardware en una GPU. Desde la perspectiva del software, el trabajo se expresa mediante *threads*, los cuales se agrupan en *thread blocks* y, a su vez, en un *grid* sobre el cual se ejecuta un *kernel*. En el hardware, estos hilos son agrupados para su ejecución bajo el modelo SIMT en *warps* o *wavefronts*, cuyas instrucciones son distribuidas entre múltiples *lanes* de ejecución. Estos recursos se organizan dentro de unidades de cómputo, denominadas comúnmente *Compute Units* (CU) o *Streaming Multiprocessors* (SM), de las cuales una GPU contiene múltiples instancias. De esta manera, el modelo de programación abstrae la distribución física del trabajo, permitiendo expresar el paralelismo

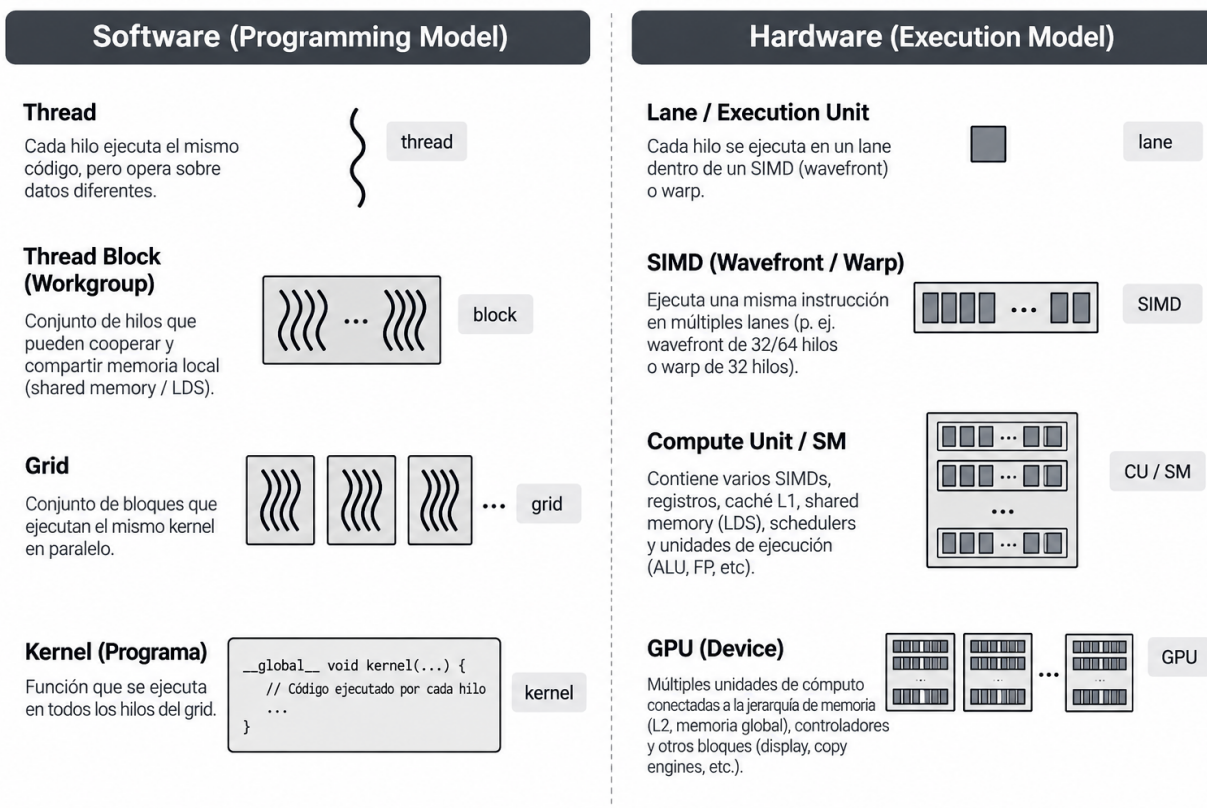


Figura 3.3: Asociación de los conceptos de SIMT con el hardware. Imagen asistida con IA

mediante hilos mientras el hardware se encarga de agruparlos, planificarlos y ejecutarlos sobre sus recursos disponibles.

SIMD y SIMT SIMT está estrechamente relacionado con SIMD (*Single Instruction, Multiple Data*), pero ambos conceptos describen el paralelismo desde perspectivas diferentes. SIMD describe principalmente una organización donde una instrucción vectorial opera explícitamente sobre varios elementos:

ADD V0, V1, V2

En SIMT, en cambio, el programador expresa múltiples hilos aparentemente independientes:

Thread 0: ADD Thread 1: ADD ... Thread N-1: ADD

El hardware agrupa estos hilos y aprovecha que ejecutan la misma instrucción para procesarlos utilizando múltiples *lanes*. De esta manera, SIMT ofrece una abstracción basada en hilos sobre hardware altamente paralelo.

Divergencia de control Una consecuencia importante de este modelo aparece cuando los hilos de un mismo grupo toman caminos diferentes en una estructura de control. Por ejemplo:

```
if (thread_id % 2 == 0)
    operation_A();
else
    operation_B();
```

Algunos *lanes* requieren ejecutar `operation_A()` mientras que otros requieren ejecutar `operation_B()`. Esta situación se conoce como *divergencia*. El hardware debe gestionar los diferentes caminos de ejecución, pudiendo ejecutar cada camino únicamente con los *lanes* correspondientes activos y posteriormente convergerlos.

Por esta razón, el modelo SIMT alcanza una mayor utilización de las unidades de ejecución cuando los hilos de un mismo *warp* o *wavefront* presentan un flujo de control similar y realizan accesos a memoria que pueden ser atendidos eficientemente.

3.3. Principios de programación usando SIMT

La programación en SIMT se orienta a que cada hilo tenga un índice de un espacio de trabajo. Podemos visualizar el espacio de trabajo como una celda de un bloque de trabajo, tal como lo ilustra la Figura 3.4. En este caso, se puede ver el espacio de trabajo como un espacio de dos dimensiones, pero puede ser de una o tres dimensiones. Con esto, cada hilo va a tener tres índices: *grid*, *bloque* e *hilo*, y además, tres dimensiones: *x*, *y* y *z*.

Una función asociada a un hilo en un espacio de dos dimensiones puede verse de la siguiente manera:

```
__global__ void kernel_function (const int pitch, int * img) {
    /* En dos dimensiones */
    int thread_pos_x = blockIdx.x * blockDim.x + threadIdx.x;
    int thread_pos_y = blockIdx.y * blockDim.y + threadIdx.y;

    /* En una dimension colapsada */
    int pixel_idx = thread_pos_x + thread_pos_y * blockDim.y * pitch;
    int value = img[pixel_idx];
}
```

Con lo anterior, tenemos tres variables de dimensión y de subdimensión por hilo:

- `threadIdx.I`: número de hilo dentro de un bloque en la subdimensión *I*
- `blockDim.I`: número de hilos total dentro de un bloque en la subdimensión *I*.
- `blockIdx.I`: número de bloque dentro de un *grid* en la subdimensión *I*.

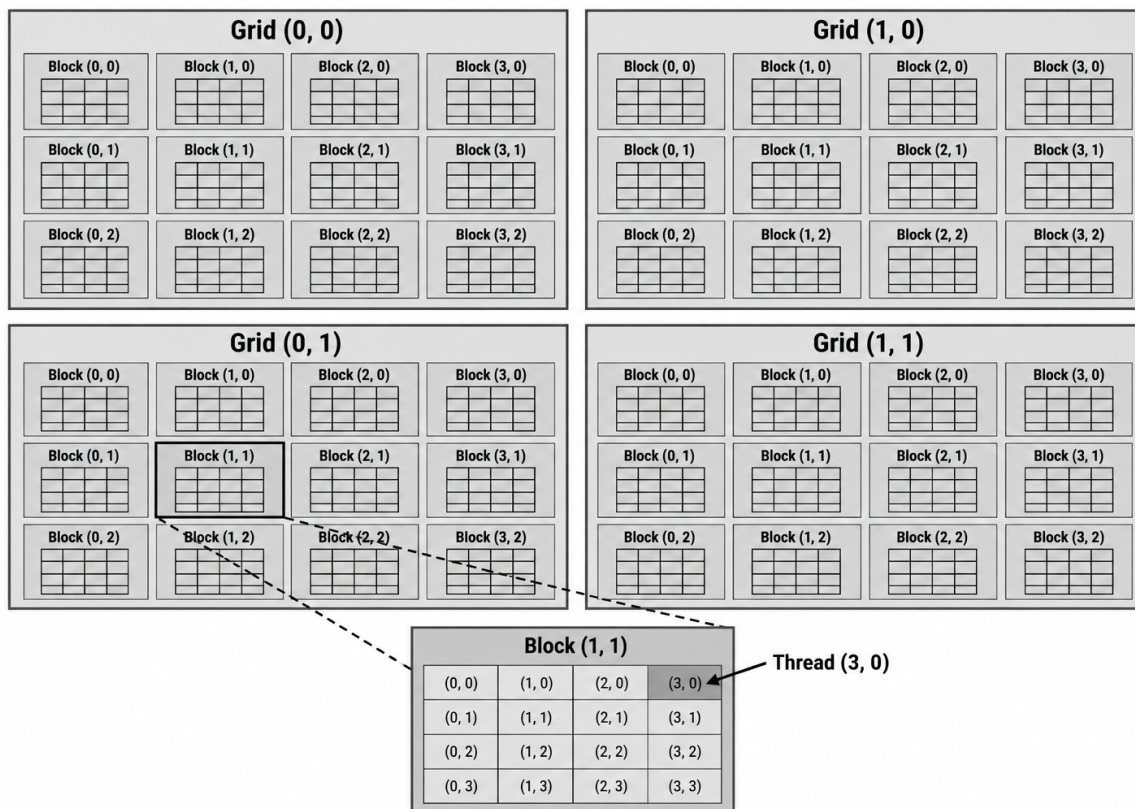


Figura 3.4: Espacio de trabajo en SIMT. Imagen asistida con IA

Consideración

El límite siempre es 1024 hilos dentro de un bloque en total.

Donde I puede ser cualquiera de las subdimensiones x, y, z.

Para invocar un kernel y lanzar a todos los hilos, se utiliza la notación de kernel o *chevron*:

```
dim3 gridsize(64, 32, 1); // orden: x y z.
dim3 blocksize(32, 32, 1); // orden: x y z.
```

```
kernel_function <<<gridsize, blocksize>>> (pitch, img);
```

Para probar el funcionamiento básico, podemos probarlo con el siguiente snippet:

```
#include <cuda_runtime.h>

__global__ void welcome() {
    int thread_pos_x = blockIdx.x * blockDim.x + threadIdx.x;
```

```
int thread_pos_y = blockIdx.y * blockDim.y + threadIdx.y;

printf("Hola desde el hilo %i, %i\n", thread_pos_x, thread_pos_y);
}

int main() {
    dim3 gridSize(2, 1, 1);
    dim3 blockSize(3, 4, 1);

    welcome<<<gridSize, blockSize>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

Recordemos que la ejecución del kernel sucede en la GPU, por lo que el CPU debería esperar que termine, mediante `cudaDeviceSynchronize()`.

Comunicación de datos

Para comunicar datos, tenemos que ser conscientes de que el GPU tiene un espacio de memoria distinto al del sistema. Esto implica que no podemos usar la memoria del sistema (CPU) en la GPU, ya que no cumple varios criterios técnicos, como el espacio de memoria, la contigüidad y la paginación. Para ello, solemos usar varios mecanismos:

Memoria manejada

La memoria manejada (*managed*) es una memoria donde el sistema operativo (mediante el driver) se vuelve consciente de los accesos y mueve la memoria desde la GPU al CPU y viceversa dependiendo de quién intenta acceder a ella. Para utilizarla, usamos la reserva con `cudaManagedAlloc` de la siguiente manera:

```
size_t size = 1024;
int *data = nullptr;

cudaMallocManaged(&data, size);
```

En práctica, tenemos el puntero en `data` listo para ser usado tanto por el CPU y el GPU, tal que podemos hacer transacciones como:

```
#include <cuda_runtime.h>
#include <stdio.h>

__global__ void kernel_function (const int pitch, int * img) {
    /* En dos dimensiones */
    int thread_pos_x = blockIdx.x * blockDim.x + threadIdx.x;
    int thread_pos_y = blockIdx.y * blockDim.y + threadIdx.y;

    /* En una dimension colapsada */
    int pixel_idx = thread_pos_x + thread_pos_y * pitch;
```

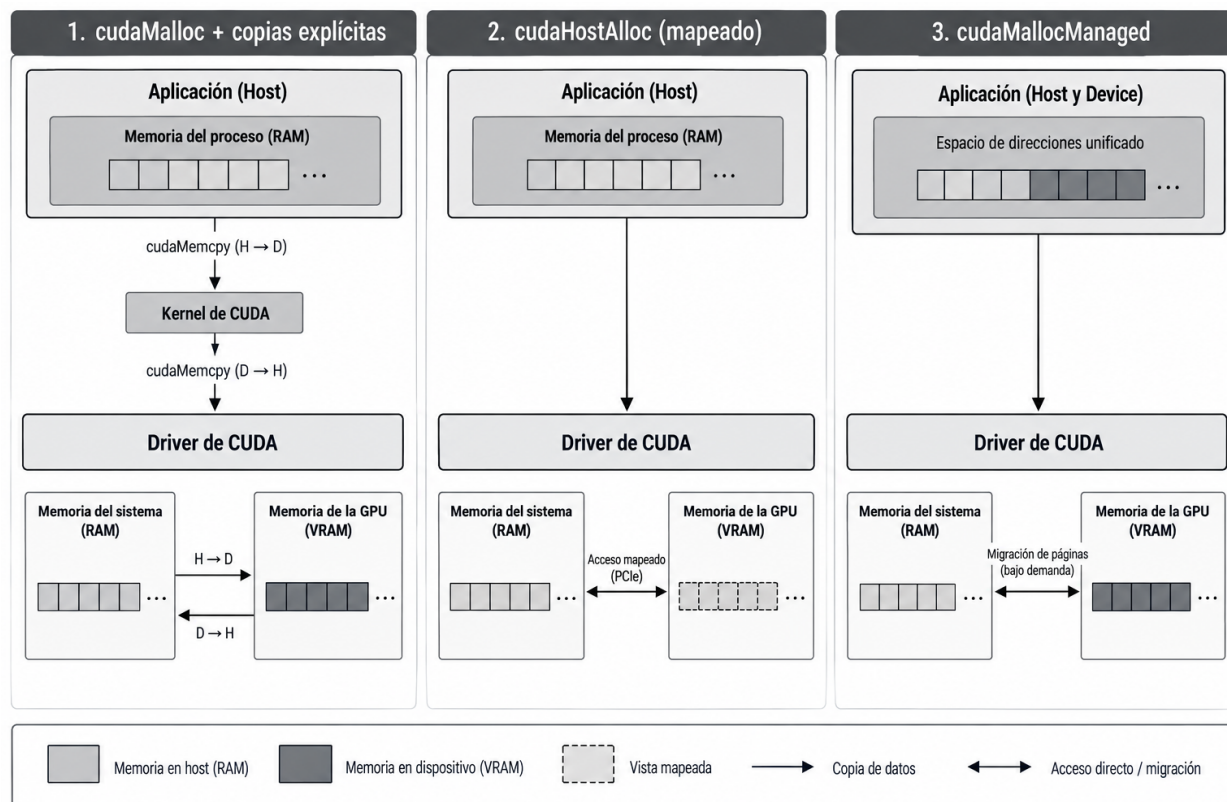


Figura 3.5: Transacciones de memoria de GPU. Imagen asistida con IA.

```

img[pixel_idx] = pixel_idx;
}

int main() {
    dim3 gridSize(2, 1, 1); // 2 bloques
    dim3 blockSize(4, 4, 1); // 16 hilos por bloque

    int width = gridSize.x * blockSize.x; // 8
    int height = gridSize.y * blockSize.y; // 4

    int pitch = width;

    size_t size = width * height * sizeof(int);
    int *data = nullptr;

    // Unico alloc
    cudaMallocManaged(&data, size);

    kernel_function<<<gridSize, blockSize>>>(pitch, data);

```

```
    cudaDeviceSynchronize();

    for (int i = 0; i < (size / sizeof(int)); i++) {
        printf("%i ", data[i]);
    }
    printf("\n");

    cudaFree(data);
}
```

Como se observa, el mismo puntero `data` puede ser enviado a la GPU y ser usado en la CPU, sin necesidad de alguna traducción explícita. Sin embargo, usar este tipo de mecanismos puede conllevar a una latencia adicional al lanzar el kernel.

Memoria mapeada

Es una alternativa a la memoria manejada, donde se hace la reserva de una sección de memoria de la CPU que el sistema operativo no puede paginar. Esto hace que la GPU pueda utilizarla directamente en un espacio de direcciones paralelo.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void kernel_function(const int pitch, int *img) {
    int thread_pos_x = blockIdx.x * blockDim.x + threadIdx.x;
    int thread_pos_y = blockIdx.y * blockDim.y + threadIdx.y;

    int pixel_idx = thread_pos_x + thread_pos_y * pitch;

    img[pixel_idx] = pixel_idx;
}

int main() {
    dim3 gridSize(2, 1, 1);
    dim3 blockSize(4, 4, 1);

    int width  = gridSize.x * blockSize.x; // 8
    int height = gridSize.y * blockSize.y; // 4

    int pitch = width;

    size_t size = width * height * sizeof(int);

    int *data_host = nullptr;
    int *data_device = nullptr;

    // Allocate page-locked host memory
    cudaHostAlloc(&data_host, size, cudaHostAllocMapped);
```

```

// Obtain the CUDA device pointer
cudaHostGetDevicePointer(&data_device, data_host, 0 );

kernel_function<<<gridsize, blocksize>>>(pitch, data_device);
cudaDeviceSynchronize();

for (int i = 0; i < (size / sizeof(int)); i++) {
    printf("%i ", data_host[i]);
}
printf("\n");

cudaFreeHost(data_host);

return 0;
}

```

En este caso, podemos observar que solo se realiza la reserva una única vez y luego se obtiene el puntero correspondiente en el espacio de direcciones de la GPU.

Memoria controlada

Es posible tener la memoria controlada explícitamente, manejando los dos espacios de direcciones por separado y decidiendo cuándo realizar la migración de datos. Esto es particularmente útil cuando se quiere mejorar el rendimiento haciendo *overlapping*. El precio a pagar es tener toda la responsabilidad de la memoria.

```

#include <stdio.h>
#include <cuda_runtime.h>

__global__ void kernel_function(const int pitch, int *img) {
    int thread_pos_x = blockIdx.x * blockDim.x + threadIdx.x;
    int thread_pos_y = blockIdx.y * blockDim.y + threadIdx.y;

    int pixel_idx = thread_pos_x + thread_pos_y * pitch;

    img[pixel_idx] = pixel_idx;
}

int main() {
    dim3 gridsize(2, 1, 1);
    dim3 blocksize(4, 4, 1);

    int width  = gridsize.x * blocksize.x; // 8
    int height = gridsize.y * blocksize.y; // 4

    int pitch = width;

    size_t size = width * height * sizeof(int);
}

```

```

int *data_host = (int *)malloc(size);
int *data_device = nullptr;

cudaMalloc(&data_device, size);

kernel_function<<<gridsize, blocksize>>>(pitch, data_device);
cudaDeviceSynchronize();

cudaMemcpy(data_host, data_device, size, cudaMemcpyDeviceToHost);

for (int i = 0; i < (size / sizeof(int)); i++) {
    printf("%i ", data_host[i]);
}
printf("\n");

cudaFree(data_device);
free(data_host);
return 0;
}

```

Todos estos modos de comunicación de datos están resumidos en la Figura 3.5.

Streams

Los *streams* permiten organizar el trabajo enviado a una GPU en **secuencias ordenadas de operaciones**. Un *stream* puede entenderse conceptualmente como una cola en la cual se insertan operaciones como transferencias de memoria, ejecución de *kernels* y otras operaciones realizadas por el dispositivo.

Las operaciones pertenecientes a un mismo *stream* mantienen un **orden de ejecución**. Por ejemplo, si se encolan una transferencia hacia el dispositivo, un *kernel* y una transferencia hacia el *host*, estas operaciones respetan las dependencias establecidas por su orden:

$$\text{H2D} \rightarrow \text{Kernel} \rightarrow \text{D2H}$$

Sin embargo, un proceso puede crear múltiples *streams*. Las operaciones pertenecientes a *streams* diferentes no poseen, en general, una dependencia implícita entre ellas. Esto permite que la GPU pueda solapar operaciones cuando existen recursos de hardware disponibles. Por ejemplo:

Stream 0: H2D₀ → Kernel₀ → D2H₀

Stream 1: H2D₁ → Kernel₁ → D2H₁

Stream 2: H2D₂ → Kernel₂ → D2H₂

De esta forma, mientras un *stream* realiza una transferencia de datos, otro puede encontrarse ejecutando un *kernel*. Incluso pueden coexistir diferentes *kernels* si la arquitectura dispone de los

recursos necesarios. Por tanto, los *streams* permiten expresar **conurrencia y dependencias** en el trabajo enviado a la GPU, mientras que el *scheduler* y el hardware determinan qué operaciones pueden ejecutarse efectivamente de manera simultánea.

```
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void kernel_function(const int pitch, int *img) {
    int thread_pos_x = blockIdx.x * blockDim.x + threadIdx.x;
    int thread_pos_y = blockIdx.y * blockDim.y + threadIdx.y;

    int pixel_idx = thread_pos_x + thread_pos_y * pitch;

    img[pixel_idx] += pixel_idx;
}

int main() {
    dim3 gridSize(2, 1, 1);
    dim3 blockSize(4, 4, 1);

    int width  = gridSize.x * blockSize.x;
    int height = gridSize.y * blockSize.y;

    int pitch = width;
    size_t size = width * height * sizeof(int);

    int *data_host = (int *)malloc(size);
    int *data_device = nullptr;

    cudaStream_t stream;
    cudaStreamCreate(&stream);

    cudaMalloc(&data_device, size);

    // Inicializar memoria en CPU
    for (int i = 0; i < width * height; i++) {
        data_host[i] = 0;
    }

    // Copia CPU -> GPU dentro del stream
    cudaMemcpyAsync(data_device, data_host, size, cudaMemcpyHostToDevice, stream)
    ↪ ;

    // Kernel ejecutado en el mismo stream
    kernel_function<<<gridSize, blockSize, 0, stream>>>(pitch, data_device);

    // Copia GPU -> CPU
    cudaMemcpyAsync(data_host, data_device, size, cudaMemcpyDeviceToHost, stream)
    ↪ ;
```

```
// Esperar a que termine todo el stream
cudaStreamSynchronize(stream);

cudaFree(data_device);
cudaStreamDestroy(stream);
free(data_host);

return 0;
}
```

3.4. Programación óptima de GPUs

La programación óptima en GPUs, al igual que en el CPU, se orienta a tomar en cuenta la microarquitectura y organización de la unidad de procesamiento.

Particularmente, la GPU es una unidad:

- Capaz de ejecutar la misma instrucción en cientos-miles de hilos de forma paralela, teniendo IPCs teóricos de más de 100 o 1000.
- Capaz de cargar datos en memoria de forma paralela por cada stream multiprocesador.

Esto implica tener dos corolarios importantes para alcanzar el potencial computacional máximo de una GPU:

- La carga de trabajo (medida en número de hilos) debe ser lo suficientemente grande para ocupar todas las unidades funcionales.
- El acceso de memoria debe aprovechar la localidad de los datos.

Adicionalmente, existe un intercambio de datos entre el CPU (host) y la GPU (device) que hay que tener en cuenta al diseñar el algoritmo de la GPU. Particularmente, la cantidad de trabajo debe ser mayor a la cantidad de datos, evitando particularmente subir/descargar datos constantemente.

En este capítulo comenzaremos por abordar detalles importantes de la programación en GPU. Mucho de este material está inspirado en el CUDA Optimisation Guide de RidgeRun [10].

Tipo y tamaño del problema

El uso de una GPU no garantiza por sí mismo una mejora en el rendimiento de una aplicación. Antes de optimizar aspectos específicos de la arquitectura, como el acceso a memoria, la ocupación o la distribución de los hilos, es necesario determinar si el problema posee características adecuadas para su ejecución paralela y si su tamaño es suficiente para aprovechar los recursos disponibles.

En términos generales, el rendimiento de una implementación en GPU depende inicialmente de dos aspectos: el **tipo de problema** y el **tamaño del problema**.

Tipo de problema

Las GPU están diseñadas para ejecutar una gran cantidad de operaciones en paralelo. Por esta razón, resultan especialmente eficientes cuando un problema puede dividirse en muchas tareas relativamente pequeñas que ejecutan operaciones similares sobre diferentes elementos de datos.

Considere una operación aplicada sobre un vector:

$$y_i = f(x_i), \quad i = 0, 1, \dots, N - 1. \quad (3.2)$$

Si el cálculo de y_i no depende del resultado obtenido para otros elementos, cada elemento puede procesarse independientemente. En una GPU, esta estructura permite asignar diferentes elementos a diferentes hilos:

$$\text{thread}_i \longrightarrow y_i = f(x_i). \quad (3.3)$$

Este tipo de problema se denomina comúnmente *data parallel*, debido a que una misma operación se aplica sobre múltiples elementos de datos.

Ejemplos típicos incluyen:

- procesamiento de imágenes y video;
- operaciones sobre vectores y matrices;
- convoluciones;
- simulaciones numéricas;
- transformaciones geométricas;
- procesamiento de señales;
- inferencia de redes neuronales;
- procesamiento de nubes de puntos.

Sin embargo, no todos los problemas presentan este nivel de independencia. Considere, por ejemplo:

$$y_i = y_{i-1} + x_i. \quad (3.4)$$

En este caso, el cálculo de y_i depende del resultado anterior. Una implementación directa presenta una dependencia secuencial:

$$y_0 \rightarrow y_1 \rightarrow y_2 \rightarrow \dots \rightarrow y_{N-1}. \quad (3.5)$$

La existencia de dependencias no implica necesariamente que el algoritmo no pueda ejecutarse eficientemente en una GPU. En algunos casos, el algoritmo puede reformularse utilizando técnicas paralelas, como reducciones, *scan*, procesamiento por bloques o algoritmos jerárquicos. Por tanto, una parte importante de la optimización consiste en identificar el **paralelismo disponible en el algoritmo** y, cuando sea posible, reformularlo para exponer una mayor cantidad de trabajo independiente.

Además del paralelismo, deben considerarse otras características del problema. Un algoritmo puede poseer miles de operaciones independientes y aun así presentar un rendimiento limitado por otros factores. Una clasificación útil consiste en distinguir entre problemas dominados por cómputo y problemas dominados por memoria.

Un problema *compute-bound* dedica una cantidad considerable de operaciones aritméticas a cada elemento de datos transferido. En estos casos, el rendimiento suele estar relacionado con la capacidad de cómputo de la GPU.

Por otro lado, un problema *memory-bound* realiza pocas operaciones por cada dato leído o escrito. En este caso, incrementar la capacidad aritmética disponible puede producir poca mejora, debido a que el rendimiento está limitado principalmente por el movimiento de datos.

Esta relación puede caracterizarse mediante la **intensidad aritmética**:

$$I = \frac{\text{operaciones realizadas}}{\text{bytes transferidos}}. \quad (3.6)$$

Una intensidad aritmética elevada generalmente favorece el aprovechamiento de las unidades de cómputo, mientras que una intensidad baja incrementa la importancia del subsistema de memoria.

Por tanto, antes de realizar una optimización específica, conviene analizar al menos tres características del problema:

1. ¿Cuánto paralelismo independiente existe?
2. ¿Cuánto trabajo realiza cada hilo?
3. ¿Cuántos datos deben moverse para realizar ese trabajo?

Tamaño del problema

Incluso cuando un algoritmo posee un alto grado de paralelismo, su ejecución en GPU puede no ser conveniente si el problema es demasiado pequeño.

La ejecución de un kernel introduce costes que no dependen directamente de la cantidad de trabajo realizado. De forma simplificada, el tiempo total puede expresarse como:

$$T_{\text{GPU}} = T_{\text{launch}} + T_{\text{transfer}} + T_{\text{kernel}} + T_{\text{sync}}, \quad (3.7)$$

donde T_{launch} corresponde al coste asociado con el lanzamiento del kernel, T_{transfer} representa los movimientos de datos requeridos, T_{kernel} corresponde al procesamiento realizado por la GPU y T_{sync} representa los posibles costes de sincronización.

Para problemas pequeños, estos costes pueden ser comparables o incluso mayores que el tiempo requerido para realizar directamente el cálculo en un CPU.

De manera conceptual:

Tamaño	GPU	CPU
Pequeño	Sobrecoste significativo	Puede ser favorable
Mediano	Mayor aprovechamiento	Depende del algoritmo
Grande	Alto paralelismo disponible	Paralelismo más limitado

El tamaño del problema también determina cuántos recursos de la GPU pueden mantenerse ocupados. Suponga que una GPU posee múltiples *Streaming Multiprocessors* (SM), cada uno capaz de mantener numerosos *warps* activos. Si únicamente se generan unos pocos bloques, parte de los SM permanecerá sin trabajo.

Por ejemplo, considere dos configuraciones:

$$N = 512 \quad (3.8)$$

y

$$N = 2^{20}. \quad (3.9)$$

Si se utilizan bloques de 256 hilos, el primer problema genera únicamente:

$$\frac{512}{256} = 2 \quad (3.10)$$

bloques. Independientemente de cuántos SM posea la GPU, solamente dos bloques estarán disponibles para ser planificados inicialmente. En cambio, para:

$$N = 2^{20}, \quad (3.11)$$

se generan:

$$\frac{2^{20}}{256} = 4096 \quad (3.12)$$

bloques, proporcionando suficiente trabajo para distribuir entre los diferentes SM y permitiendo al planificador mantener una mayor cantidad de recursos ocupados.

Por esta razón, no solamente importa que un algoritmo sea paralelizable. También debe existir **suficiente paralelismo** para alimentar la arquitectura.

Granularidad del trabajo

El tamaño del problema está estrechamente relacionado con la **granularidad**, es decir, la cantidad de trabajo asignada a cada hilo. Una estrategia sencilla consiste en asignar un elemento por hilo:

$$\text{thread}_i \longrightarrow x_i. \quad (3.13)$$

Sin embargo, también es posible asignar múltiples elementos a cada hilo:

$$\text{thread}_i \longrightarrow \{x_i, x_{i+T}, x_{i+2T}, \dots\}, \quad (3.14)$$

donde T corresponde al número total de hilos utilizados.

Una granularidad demasiado pequeña puede incrementar proporcionalmente los costes asociados con la planificación, el lanzamiento de kernels y otras operaciones auxiliares. Por el contrario, una granularidad excesivamente grande puede reducir la cantidad de paralelismo disponible. Existe, por tanto, un compromiso entre:

trabajo por hilo	$\longleftrightarrow$	cantidad de hilos
------------------	-----------------------	-------------------

(3.15)

La configuración óptima depende de la arquitectura, del algoritmo y de los recursos utilizados por cada hilo.

Paralelismo disponible y utilización de la GPU

Un concepto importante es distinguir entre que un problema sea *paralelizable* y que posea suficiente paralelismo para utilizar eficientemente una GPU.

Por ejemplo:

$$32 \text{ tareas independientes} \quad (3.16)$$

constituyen un problema paralelizable. Sin embargo, para una GPU moderna esta cantidad de trabajo puede ser insuficiente para utilizar una fracción significativa de sus recursos. En contraste:

$$10^6 \text{ tareas independientes} \quad (3.17)$$

permite distribuir el trabajo entre miles de hilos y mantener múltiples bloques y *warps* disponibles para ejecución.

Esta abundancia de trabajo también es importante para ocultar latencias. Cuando un *warp* debe esperar, por ejemplo, por una operación de memoria, el SM puede seleccionar otro *warp* listo para ejecutar. Este mecanismo requiere que exista suficiente trabajo disponible.

Optimización del manejo de memoria

El rendimiento de una GPU no depende únicamente de su capacidad para ejecutar una gran cantidad de operaciones en paralelo. Una parte importante del tiempo y de la energía consumida por una aplicación está asociada con el **movimiento de datos**. Por esta razón, una optimización eficiente debe procurar que los datos sean transferidos la menor cantidad de veces posible y que, cuando sea necesario acceder a ellos, se utilice eficientemente el ancho de banda disponible.

El problema puede analizarse en diferentes niveles. Primero, debe considerarse cómo los hilos de un *warp* acceden a la memoria. Posteriormente, debe analizarse dónde se almacenan los datos durante la ejecución de una secuencia de operaciones. Finalmente, debe seleccionarse adecuadamente el tipo de memoria utilizado y determinar cuándo es necesario mover, reutilizar o compartir los datos.

Acceso coalesced a memoria

Cuando los hilos de un *warp* ejecutan una instrucción de acceso a memoria global, las solicitudes individuales son agrupadas por el hardware en un conjunto de transacciones de memoria. Por esta razón, la distribución de las direcciones solicitadas por los hilos tiene un impacto directo sobre la eficiencia del acceso.

Considere un arreglo:

$$x = \{x_0, x_1, x_2, \dots, x_{N-1}\}. \quad (3.18)$$

Una asignación natural consiste en hacer que cada hilo acceda a un elemento consecutivo:

$$\text{thread}_i \longrightarrow x_i. \quad (3.19)$$

Por ejemplo:

Hilo	0	1	2	3	4	5	...	31
Dato	x_0	x_1	x_2	x_3	x_4	x_5	...	x_{31}

En este caso, los hilos acceden a posiciones contiguas de memoria. El hardware puede agrupar estas solicitudes en un número reducido de transacciones. Este patrón se conoce como **acceso coalesced**.

En contraste, considere un acceso con un *stride* S :

$$\text{thread}_i \longrightarrow x_{Si}. \quad (3.20)$$

Para $S = 4$:

Hilo	0	1	2	3	4	...
Dato	x_0	x_4	x_8	x_{12}	x_{16}	...

Los elementos solicitados se encuentran ahora separados en memoria. Como consecuencia, pueden ser necesarias más transacciones para atender las solicitudes del *warp*. Parte de los datos transferidos en cada transacción puede no ser utilizada, reduciendo el ancho de banda efectivo.

Por tanto, una regla fundamental al diseñar un kernel es procurar que:

Consideración

hilos consecutivos $\rightarrow$ direcciones de memoria consecutivas

cuando la estructura del algoritmo lo permita.

Esto es particularmente importante en operaciones sobre imágenes, matrices, tensores y otros arreglos multidimensionales. La forma en que estos datos son linealizados en memoria debe considerarse al determinar qué dimensión del problema se asigna a cada dimensión de los hilos.

Organización de las estructuras de datos

La coalescencia también puede depender de cómo se organizan las estructuras de datos. Considere un conjunto de puntos tridimensionales:

```
struct Point {
    float x;
    float y;
    float z;
};
```

Una representación denominada *Array of Structures* (AoS) utiliza:

```
Point points[N];
```

produciendo conceptualmente:

$$[x_0, y_0, z_0][x_1, y_1, z_1][x_2, y_2, z_2] \dots \quad (3.21)$$

Si un kernel solamente necesita procesar la coordenada x , los accesos presentan un salto entre elementos consecutivos.

Una alternativa es utilizar una organización *Structure of Arrays* (SoA):

```
struct Points {
    float x[N];
    float y[N];
    float z[N];
};
```

cuya distribución conceptual es:

$$[x_0, x_1, x_2, \dots][y_0, y_1, y_2, \dots][z_0, z_1, z_2, \dots]. \quad (3.22)$$

Ahora los hilos consecutivos que procesan la coordenada x acceden a posiciones consecutivas. Dependiendo del algoritmo, una reorganización de AoS hacia SoA puede favorecer accesos coalesced y mejorar el aprovechamiento del ancho de banda.

No obstante, SoA no es universalmente superior a AoS. La estructura apropiada depende de cuáles campos sean utilizados conjuntamente y del patrón de acceso de los kernels.

Modelo de intercambio de datos

Previo a continuar con la memoria fuera de los SMs, es importante conocer cómo llegan los datos desde la CPU hasta la GPU. La Figura 3.6 ilustra el proceso, en el que se deben copiar explícitamente los datos (incluso en los modelos implícitos). Estos datos viajan desde la memoria DDR del sistema, pasan por PCIe y llegan a la memoria GDDR de la GPU.

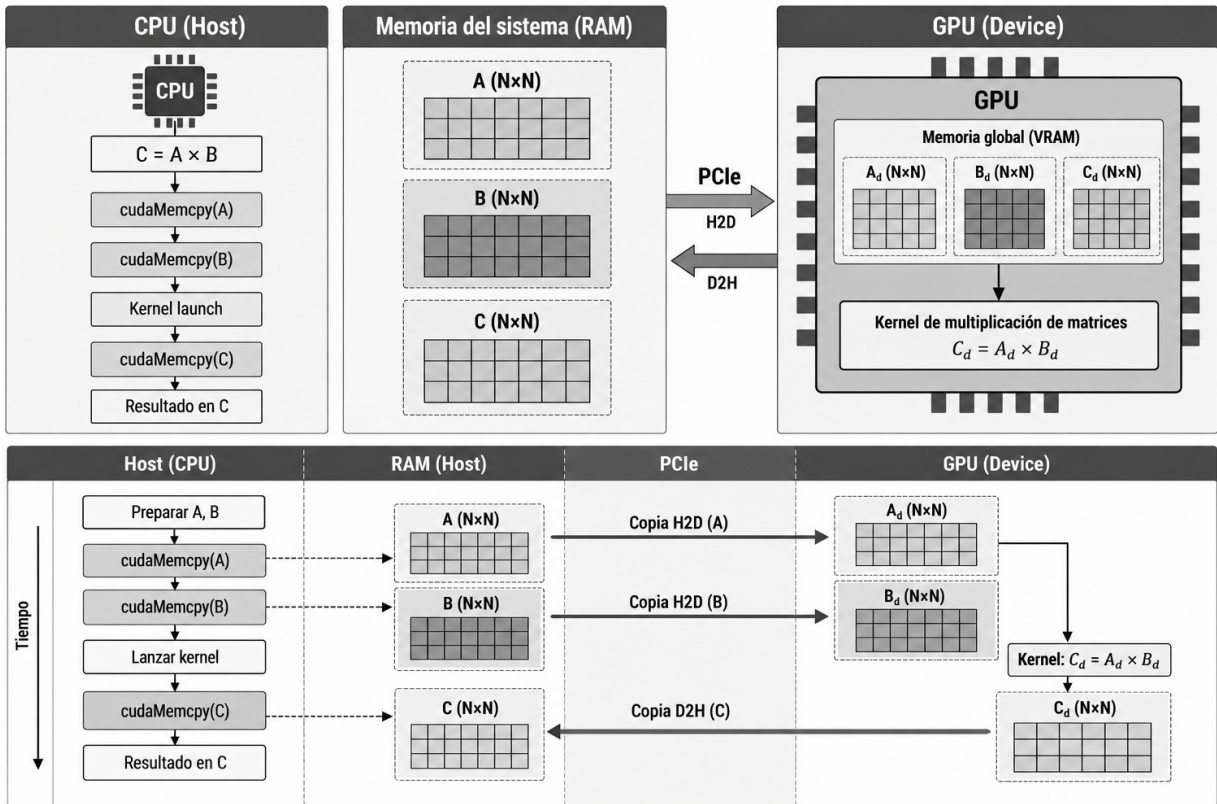


Figura 3.6: Comunicación host-device dentro de la GPU

El movimiento de datos puede convertirse en un factor limitante cuando el ancho de banda de la interconexión es considerablemente menor que el disponible dentro del dispositivo. La Tabla 3.1 muestra valores teóricos representativos.

Tabla 3.1: Anchos de banda teóricos aproximados de diferentes tecnologías.

Tecnología	Velocidad típica	Ancho de banda
PCIe 3.0 x16	8 GT/s	~15.8 GB/s por dirección
PCIe 4.0 x16	16 GT/s	~31.5 GB/s por dirección
PCIe 5.0 x16	32 GT/s	~63 GB/s por dirección
DDR4-3200	3200 MT/s	25.6 GB/s por canal
DDR5-4800	4800 MT/s	38.4 GB/s por módulo*
DDR5-6400	6400 MT/s	51.2 GB/s por módulo*
GDDR5	5–8 Gb/s/pin	~160–256 GB/s**
GDDR6	14–24 Gb/s/pin	~448–768 GB/s**
GDDR7	28–32+ Gb/s/pin	~896–1024+ GB/s**

* Valores agregados para los dos subcanales de 32 bits de un módulo DDR5, sin considerar ECC.

** Ejemplo suponiendo un bus de memoria de 256 bits. El ancho de banda real depende del ancho del bus implementado por la GPU.

Estos valores permiten observar una diferencia importante. Una GPU puede disponer internamente de varios cientos de GB/s, e incluso superar 1 TB/s de ancho de banda hacia su memoria, mientras que una transferencia mediante PCIe proporciona un ancho de banda considerablemente menor.

Por ejemplo, una GPU con GDDR6 de 16 Gb/s y un bus de 256 bits posee teóricamente:

$$BW = \frac{16 \times 10^9 \times 256}{8} = 512 \text{ GB/s.} \quad (3.23)$$

En contraste, PCIe 4.0 x16 proporciona aproximadamente 31.5 GB/s por dirección. Por tanto, el ancho de banda de la memoria local de la GPU puede ser más de un orden de magnitud superior al disponible para transferir datos entre CPU y GPU.

Residencia de los datos

Como se puede intuir de lo anterior, la optimización de memoria no debe analizarse únicamente dentro de un kernel. También debe considerarse el movimiento de los datos durante la ejecución

completa de la aplicación.

Si las operaciones se ejecutan consecutivamente sobre los mismos datos, resulta preferible mantenerlos residentes en memoria accesible por la GPU:

$$\boxed{\text{CPU} \xrightarrow{\text{transferencia}} \text{GPU} \rightarrow A \rightarrow B \rightarrow C \xrightarrow{\text{transferencia}} \text{CPU}} \quad (3.24)$$

De esta forma, la transferencia inicial y final se amortizan sobre una mayor cantidad de trabajo.

La optimización debe realizarse, por tanto, considerando el **flujo completo de datos** y no solamente el rendimiento de kernels individuales.

Reutilización de buffers

Mantener los datos residentes también implica evitar asignaciones y liberaciones innecesarias.

Por ejemplo, una aplicación que procesa continuamente imágenes podría realizar para cada imagen:

```
cudaMalloc(...)
cudaMemcpy(...)
kernel(...)
cudaMemcpy(...)
cudaFree(...)
```

Sin embargo, si todas las imágenes poseen dimensiones compatibles, es posible reservar los buffers una vez:

```
cudaMalloc(...)

for (...) {
    cudaMemcpy(...)
    kernel(...)
    cudaMemcpy(...)
}

cudaFree(...)
```

El mismo principio puede aplicarse a buffers intermedios. Un *pipeline* de procesamiento:

$$\text{Input} \rightarrow K_1 \rightarrow K_2 \rightarrow K_3 \rightarrow \text{Output} \quad (3.25)$$

puede reservar al inicio todos los buffers requeridos y reutilizarlos durante múltiples iteraciones.

En aplicaciones de ejecución continua, como procesamiento de video, percepción robótica o inferencia, esta estrategia evita introducir operaciones repetitivas de gestión de memoria en la ruta crítica.

Kernel fusion

Mantener los datos en el dispositivo evita transferencias entre CPU y GPU, pero todavía pueden existir movimientos innecesarios dentro de la propia memoria de la GPU.

Considere dos operaciones consecutivas:

$$y_i = f(x_i), \quad (3.26)$$

$$z_i = g(y_i). \quad (3.27)$$

Utilizando dos kernels independientes:

$$\text{Global Memory} \rightarrow K_1 \rightarrow \text{Global Memory} \rightarrow K_2 \rightarrow \text{Global Memory}. \quad (3.28)$$

El resultado y_i debe escribirse en memoria global y posteriormente volver a leerse.

Si ambas operaciones pueden combinarse, es posible implementar:

$$z_i = g(f(x_i)), \quad (3.29)$$

mediante un único kernel:

$$\text{Global Memory} \rightarrow K_{1+2} \rightarrow \text{Global Memory}. \quad (3.30)$$

El valor intermedio puede permanecer en un registro:

```
float tmp = f(input[i]);
output[i] = g(tmp);
```

Esta técnica se conoce como **kernel fusion**. Sus posibles beneficios incluyen:

- reducir accesos a memoria global;
- eliminar buffers intermedios;
- reducir el número de lanzamientos de kernels;
- incrementar la intensidad aritmética;
- mejorar la localidad de los datos.

Sin embargo, fusionar kernels no siempre produce una mejora. Un kernel más complejo puede requerir más registros, memoria compartida u otros recursos. Esto puede reducir la cantidad de bloques o *warps* que pueden permanecer activos simultáneamente.

Por tanto, existe un compromiso:

$$\boxed{\text{menos movimiento de datos} \longleftrightarrow \text{mayor consumo de recursos por kernel}} \quad (3.31)$$

La decisión debe tomarse considerando el comportamiento global de la aplicación.

Jerarquía de memoria

Una GPU proporciona diferentes espacios de memoria con distintas propiedades. De forma conceptual, estos pueden organizarse como:

Memoria	Alcance	Latencia relativa	Capacidad
Registros	Hilo	Muy baja	Muy pequeña
Shared Memory	Bloque	Baja	Pequeña
Caché	SM/GPU	Baja-media	Intermedia
Global Memory	GPU	Alta	Grande
Host Memory	CPU	Muy alta desde GPU	Grande

La estrategia general consiste en acercar los datos utilizados frecuentemente hacia memorias más próximas a las unidades de ejecución.

Por ejemplo, si múltiples hilos de un bloque reutilizan los mismos datos, puede resultar conveniente:

$$\text{Global Memory} \rightarrow \text{Shared Memory} \rightarrow \text{múltiples operaciones.} \quad (3.32)$$

El coste de cargar los datos desde memoria global se amortiza mediante su reutilización.

Un ejemplo clásico ocurre en multiplicación de matrices mediante *tiling*. Un bloque carga pequeñas regiones de las matrices en memoria compartida y realiza múltiples operaciones sobre ellas antes de solicitar nuevos datos desde memoria global.

Registros y presión de registros

Los registros representan uno de los recursos de almacenamiento más rápidos disponibles para un hilo. Variables locales y resultados temporales pueden mantenerse en registros durante la ejecución de un kernel.

Por ejemplo:

```
float a = input[i];
float b = a * scale;
float c = b + offset;
output[i] = c;
```

Los valores intermedios pueden permanecer en registros sin necesidad de almacenarse en memoria global.

Sin embargo, los registros constituyen un recurso limitado del SM. Si cada hilo utiliza una gran cantidad de registros, puede disminuir la cantidad de *warps* que pueden residir simultáneamente en el SM.

Además, cuando la demanda de registros supera los recursos disponibles, algunos valores pueden ser desplazados hacia memoria mediante un fenómeno denominado *register spilling*, introduciendo accesos adicionales a memoria y potencialmente reduciendo el rendimiento.

Por esta razón, maximizar el uso de registros no constituye por sí mismo una estrategia de optimización. Debe existir un equilibrio entre almacenamiento local rápido y paralelismo disponible.

Memoria compartida

La *shared memory* permite que los hilos pertenecientes a un mismo bloque compartan datos utilizando una memoria de baja latencia ubicada cerca de las unidades de ejecución.

Un patrón frecuente consiste en:

1. cargar datos desde memoria global;
2. almacenarlos temporalmente en memoria compartida;
3. sincronizar los hilos del bloque;
4. reutilizar los datos múltiples veces;
5. escribir el resultado final en memoria global.

Conceptualmente:

$$\boxed{\text{Global} \rightarrow \text{Shared} \rightarrow \underbrace{\text{Compute} \rightarrow \text{Compute} \rightarrow \text{Compute}}_{\text{reutilización}} \rightarrow \text{Global}} \quad (3.33)$$

La memoria compartida resulta particularmente útil cuando existe reutilización de datos entre los hilos. Si un dato solamente será utilizado una vez, copiarlo primero hacia memoria compartida puede introducir trabajo adicional sin proporcionar un beneficio.

Memoria del host y transferencias

Cuando CPU y GPU poseen memorias físicamente separadas, las transferencias entre ambos espacios pueden representar un coste considerable.

Una operación convencional puede requerir:

$$\text{Host Memory} \xrightarrow{\text{copia}} \text{Device Memory}. \quad (3.34)$$

Por esta razón, una estrategia general de optimización consiste en minimizar la frecuencia de estas transferencias.

CUDA proporciona además diferentes mecanismos para gestionar la interacción entre memoria del host y del dispositivo, entre ellos:

- memoria convencional del host;
- *page-locked* o *pinned memory*;
- memoria mapeada;
- *Unified Memory*;
- memoria explícitamente reservada en el dispositivo.

Estas alternativas no deben interpretarse simplemente como memorias “más rápidas” o “más lentas”. Cada una modifica aspectos como quién administra el movimiento de datos, si pueden realizarse transferencias asíncronas, si CPU y GPU pueden direccionar el mismo almacenamiento y dónde se encuentran físicamente los datos.

Por tanto, la selección depende de la arquitectura del sistema y del patrón de acceso de la aplicación.

Arquitecturas discretas e integradas

El coste de movimiento de memoria depende también de la arquitectura física del sistema.

En una GPU discreta, CPU y GPU generalmente poseen memorias físicas independientes:

$$\boxed{\text{CPU}} \longleftrightarrow \boxed{\text{Interconexión}} \longleftrightarrow \boxed{\text{GPU}} \quad (3.35)$$

$$\text{RAM} \qquad \qquad \text{VRAM}. \quad (3.36)$$

Mover información entre ambos dominios puede requerir una transferencia explícita a través de una interconexión, como PCI Express.

En un sistema integrado, CPU y GPU pueden compartir la misma memoria física:



Esto modifica considerablemente el coste y las estrategias asociadas al movimiento de datos. Por ejemplo, ciertas aplicaciones pueden utilizar memoria mapeada para permitir que CPU y GPU trabajen sobre el mismo almacenamiento físico, evitando una copia explícita.

No obstante, compartir memoria física no significa que todos los accesos posean el mismo coste. Las jerarquías de caché, coherencia, ancho de banda y mecanismos de traducción de direcciones continúan afectando el rendimiento.

Streams y asincronismo

Hasta este punto, las optimizaciones se han enfocado en reducir el movimiento de datos y mejorar la forma en que estos son accedidos. Sin embargo, algunas operaciones costosas, como las transferencias entre CPU y GPU, no siempre pueden eliminarse. En estos casos, una estrategia adicional consiste en **ocultar su latencia mediante la ejecución concurrente de otras operaciones**.

CUDA permite organizar el trabajo mediante *streams*. Un stream puede entenderse como una **secuencia ordenada de operaciones** que serán ejecutadas respetando el orden en que fueron enviadas.

Por ejemplo:

$$S_0 : H2D(A) \rightarrow K_1 \rightarrow K_2 \rightarrow D2H(C). \tag{3.38}$$

Las operaciones pertenecientes al mismo stream mantienen dependencias de orden. Sin embargo, operaciones pertenecientes a diferentes streams pueden potencialmente ejecutarse concurrentemente:

$$\begin{array}{ll}
 S_0 : & H2D(A_0) \rightarrow K(A_0) \rightarrow D2H(A_0) \\
 S_1 : & H2D(A_1) \rightarrow K(A_1) \rightarrow D2H(A_1).
 \end{array} \tag{3.39}$$

Esto permite aprovechar simultáneamente diferentes recursos disponibles en el sistema, como los motores de transferencia y las unidades de cómputo de la GPU.

Ejecución síncrona

Considere el procesamiento de tres bloques de datos. Una implementación completamente secuencial puede realizar:

$$\begin{aligned}
H2D_0 &\rightarrow K_0 \rightarrow D2H_0 \rightarrow \\
H2D_1 &\rightarrow K_1 \rightarrow D2H_1 \rightarrow \\
H2D_2 &\rightarrow K_2 \rightarrow D2H_2.
\end{aligned} \tag{3.40}$$

Durante una transferencia, algunos recursos de cómputo pueden permanecer sin utilizar. De forma similar, durante la ejecución del kernel, los recursos utilizados para realizar transferencias pueden permanecer disponibles.

El tiempo aproximado para procesar N elementos de trabajo sería:

$$T_{\text{seq}} \approx N(T_{H2D} + T_K + T_{D2H}). \tag{3.41}$$

Solapamiento de cómputo y comunicación

Utilizando múltiples streams, el conjunto de datos puede dividirse en partes o *chunks*. Mientras la GPU procesa un bloque, pueden realizarse transferencias asociadas con otros bloques.

Una situación ideal puede visualizarse como:

Tiempo --->

```

H2D:    [ A0 ] [ A1 ] [ A2 ] [ A3 ]
GPU:      [ K0 ] [ K1 ] [ K2 ] [ K3 ]
D2H:      [ R0 ] [ R1 ] [ R2 ] [ R3 ]

```

Después de llenar el pipeline, diferentes recursos trabajan simultáneamente.

Conceptualmente:

$$\boxed{H2D(n+1) \parallel Kernel(n) \parallel D2H(n-1)} \tag{3.42}$$

donde el símbolo $\parallel$ representa operaciones que potencialmente pueden solaparse.

El objetivo no es necesariamente reducir la latencia individual de una transferencia o de un kernel, sino incrementar el **throughput** del sistema completo.

Transferencias asíncronas

CUDA proporciona operaciones asíncronas como:

```

cudaMemcpyAsync(..., stream);
kernel<<<grid, block, 0, stream>>>(...);

```

Por ejemplo:

```

cudaMemcpyAsync(d_in, h_in, size,
                cudaMemcpyHostToDevice, stream);

kernel<<<grid, block, 0, stream>>>(d_in, d_out);

cudaMemcpyAsync(h_out, d_out, size,
                cudaMemcpyDeviceToHost, stream);

```

La llamada desde el CPU puede retornar antes de que la operación haya finalizado. El host puede entonces continuar ejecutando otras instrucciones o enviar trabajo adicional.

Esto introduce una distinción importante:

$$\boxed{\text{operación enviada} \neq \text{operación terminada}} \quad (3.43)$$

Por tanto, el uso de asincronismo requiere administrar explícitamente las dependencias entre operaciones.

Pinned memory

Para lograr un solapamiento efectivo de ciertas transferencias entre host y dispositivo, normalmente se utiliza memoria del host *page-locked* o *pinned*.

Esta puede reservarse mediante:

```
cudaHostAlloc(&ptr, size, cudaHostAllocDefault);
```

o registrando memoria existente mediante:

```
cudaHostRegister(ptr, size, flags);
```

Al impedir que estas páginas sean desplazadas por el sistema operativo, el dispositivo puede realizar transferencias DMA de manera más directa.

Sin embargo, la memoria *pinned* constituye un recurso que debe utilizarse cuidadosamente. Reservar cantidades excesivas puede perjudicar la administración de memoria del sistema operativo.

Por tanto, una estrategia común consiste en reservar un conjunto limitado de buffers y reutilizarlos durante toda la ejecución.

Procesamiento mediante múltiples streams

Suponga que un conjunto grande de datos se divide en N regiones. Cada región puede procesarse mediante un stream diferente:

```

for (int i = 0; i < N; ++i) {

    cudaMemcpyAsync(d_in[i], h_in[i], size,
                   cudaMemcpyHostToDevice,
                   stream[i]);

    kernel<<<grid, block, 0, stream[i]>>>(
        d_in[i], d_out[i]);

    cudaMemcpyAsync(h_out[i], d_out[i], size,
                   cudaMemcpyDeviceToHost,
                   stream[i]);
}

```

La independencia entre streams proporciona al hardware oportunidades para solapar las operaciones.

No obstante,

$$\boxed{\text{asincronismo} \neq \text{ejecución paralela garantizada}} \quad (3.44)$$

Los streams expresan dependencias y oportunidades de concurrencia. La ejecución simultánea efectiva depende de los recursos disponibles, las capacidades del dispositivo y la cantidad de recursos utilizados por cada kernel.

Eventos

Cuando existen dependencias entre streams, no siempre es necesario sincronizar toda la GPU. CUDA proporciona *events* que permiten expresar dependencias más específicas.

Por ejemplo:

```

kernelA<<<..., streamA>>>(...);

cudaEventRecord(event, streamA);

cudaStreamWaitEvent(streamB, event);

kernelB<<<..., streamB>>>(...);

```

La relación resultante es:

$$K_A \rightarrow \text{event} \rightarrow K_B. \quad (3.45)$$

El resto de las operaciones independientes puede continuar ejecutándose.

Esta idea conduce a un principio importante:

$$\boxed{\text{Sincronizar únicamente cuando exista una dependencia}} \quad (3.46)$$

Sincronización y comunicación entre hilos

La ejecución paralela requiere mecanismos que permitan coordinar las operaciones realizadas por diferentes hilos. Estos mecanismos son necesarios cuando existe una dependencia de datos o cuando múltiples hilos cooperan para producir un resultado.

Sin embargo, la sincronización tiene un coste. Un hilo que alcanza una barrera puede tener que esperar a que otros hilos completen su trabajo. Por esta razón, una optimización importante consiste en reducir tanto la frecuencia como el alcance de las sincronizaciones.

Dependencias entre hilos

Considere dos hilos donde uno produce un dato utilizado por otro:

$$\text{Thread}_0 : \quad x \leftarrow f(a), \quad (3.47)$$

$$\text{Thread}_1 : \quad y \leftarrow g(x). \quad (3.48)$$

Existe una dependencia:

$$\text{produce}(x) \rightarrow \text{consume}(x). \quad (3.49)$$

El segundo hilo no debe consumir el valor hasta que este haya sido producido y sea visible para el consumidor.

Este tipo de situación requiere mecanismos de comunicación y sincronización adecuados al alcance de los hilos involucrados.

Sincronización dentro de un bloque

Los hilos pertenecientes al mismo bloque pueden cooperar utilizando memoria compartida. Considere:

```
__shared__ float data[256];

data[threadIdx.x] = input[i];

__syncthreads();

float value = data[...];
```

La operación:

```
__syncthreads();
```

actúa como una barrera para los hilos del bloque. Ningún hilo continúa más allá de la barrera hasta que los hilos participantes hayan alcanzado ese punto. Conceptualmente:

```
Thread 0: ---- trabajo ---- | barrera | ---- continúa
Thread 1: ----- trabajo -- | barrera | ---- continúa
Thread 2: -- trabajo ----- | barrera | ---- continúa
Thread 3: ----- trabajo --- | barrera | ---- continúa
                        __syncthreads()
```

Esto permite implementar un patrón común:

$$\boxed{\text{Load} \rightarrow \text{Synchronize} \rightarrow \text{Compute}} \quad (3.50)$$

y posteriormente:

$$\boxed{\text{Compute} \rightarrow \text{Synchronize} \rightarrow \text{Reuse}} \quad (3.51)$$

Comunicación mediante memoria compartida

La memoria compartida permite implementar comunicación explícita entre los hilos de un bloque:

$$\text{Thread}_i \rightarrow \text{Shared Memory} \rightarrow \text{Thread}_j. \quad (3.52)$$

Por ejemplo:

```
__shared__ float buffer[256];

buffer[threadIdx.x] = value;

__syncthreads();

float neighbour =
    buffer[(threadIdx.x + 1) % blockDim.x];
```

Este mecanismo aparece frecuentemente en algoritmos de reducción, convolución, multiplicación de matrices, histogramas y procesamiento por *tiles*.

Sincronización a nivel de warp

En ocasiones, la cooperación solamente involucra los hilos pertenecientes a un mismo *warp*. CUDA proporciona primitivas específicas para este nivel, como:

```
__syncwarp();
```

Reducir el alcance de la sincronización puede evitar que hilos que no participan en una dependencia tengan que esperar. De forma conceptual:

$$\text{warp synchronization} \subset \text{block synchronization}. \quad (3.53)$$

Si una dependencia solamente involucra un warp, sincronizar todo el bloque puede introducir restricciones innecesarias.

Warp shuffle

Los hilos pertenecientes a un warp también pueden intercambiar determinados valores directamente mediante instrucciones *shuffle*. Por ejemplo:

```
float other = __shfl_down_sync(mask, value, offset);
```

Conceptualmente:

$$\text{registro}_i \longrightarrow \text{registro}_j. \quad (3.54)$$

Esto permite intercambiar valores sin realizar explícitamente:

$$\text{registro} \rightarrow \text{shared memory} \rightarrow \text{registro}. \quad (3.55)$$

Un ejemplo común es una reducción dentro de un warp:

```
for (int offset = warpSize / 2;
     offset > 0;
     offset /= 2) {

    value += __shfl_down_sync(
        0xffffffff, value, offset);
}
```

El patrón de comunicación evoluciona aproximadamente como:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1. \quad (3.56)$$

Este tipo de comunicación puede ser considerablemente eficiente debido a que los valores permanecen en registros.

Operaciones atómicas

Cuando múltiples hilos modifican una misma posición de memoria, puede producirse una condición de carrera.

Por ejemplo:

```
counter++;
```

no constituye necesariamente una operación indivisible. Múltiples hilos pueden leer el mismo valor y posteriormente sobrescribir resultados entre sí.

CUDA proporciona operaciones atómicas:

```
atomicAdd(&counter, 1);
```

que garantizan que la modificación correspondiente se realice de manera atómica. Estas operaciones son útiles para implementar contadores, histogramas, acumulaciones y otras estructuras compartidas. Sin embargo, cuando muchos hilos compiten por una misma dirección:

$$\text{Thread}_0 \searrow \quad (3.57)$$

$$\text{Thread}_1 \rightarrow \boxed{x} \leftarrow \text{Thread}_2 \quad (3.58)$$

$$\text{Thread}_3 \nearrow \quad (3.59)$$

se genera contención y parte del paralelismo efectivo puede perderse. Una estrategia frecuente consiste en realizar primero acumulaciones locales:

$$\text{Thread} \rightarrow \text{Warp} \rightarrow \text{Block} \rightarrow \text{Global}, \quad (3.60)$$

reduciendo progresivamente los resultados antes de ejecutar operaciones atómicas globales.

Sincronización entre bloques

Los bloques de un kernel convencional deben diseñarse, en general, suponiendo que pueden ejecutarse independientemente. No debe asumirse que:

$$\text{Block}_0 \rightarrow \text{Block}_1 \rightarrow \text{Block}_2 \quad (3.61)$$

será el orden de ejecución.

El planificador puede ejecutar bloques en diferentes SM y en un orden que no está determinado por su identificador. Una forma sencilla de establecer una dependencia global consiste en finalizar un kernel e iniciar otro:

$$K_1 \rightarrow \boxed{\text{frontera global}} \rightarrow K_2. \quad (3.62)$$

El segundo kernel puede utilizar los resultados producidos por el primero una vez satisfechas las dependencias correspondientes.

Existen mecanismos más avanzados para cooperación entre bloques, particularmente mediante *Cooperative Groups*, que permiten expresar patrones de cooperación con diferentes alcances. Estos mecanismos serán analizados posteriormente.

Coste de la sincronización

Una barrera puede hacer que hilos rápidos esperen por aquellos que requieren más tiempo:

```
Thread 0: ----|.....|----
Thread 1: -----|.....|----
Thread 2: -----|....|----
Thread 3: -----|-----
                barrera
```

Por tanto, introducir más sincronización no hace que un programa paralelo sea necesariamente mejor o más seguro. La sincronización debe representar dependencias reales del algoritmo.

Como principio de optimización:

Consideración

Sincronizar lo mínimo necesario

Esto puede interpretarse en dos dimensiones:

1. **Frecuencia:** evitar barreras innecesarias.
2. **Alcance:** sincronizar únicamente los hilos que necesitan cooperar.

Por ejemplo, si una dependencia involucra únicamente hilos de un warp, puede ser innecesario sincronizar todo el bloque. Si dos kernels son independientes, tampoco es necesario sincronizar el dispositivo entre ambos.

Niveles de comunicación y sincronización

Los diferentes mecanismos pueden resumirse según el alcance de la comunicación:

Alcance	Comunicación	Sincronización
Hilo	Registros	Secuencia de instrucciones
Warp	Shuffle / registros	<code>__syncwarp()</code>
Bloque	Shared memory	<code>__syncthreads()</code>
GPU	Global memory	Kernels / mecanismos globales
Streams	Global memory	Events
CPU-GPU	Memoria / transferencias	Streams, events, host sync

La selección del mecanismo de comunicación debe procurar utilizar el menor alcance que permita resolver correctamente la dependencia:

$$\text{Thread} \rightarrow \text{Warp} \rightarrow \text{Block} \rightarrow \text{Device} \rightarrow \text{Host} \quad (3.63)$$

A medida que aumenta el alcance de la comunicación, generalmente aumenta también su coste y la cantidad de ejecución potencialmente afectada por la sincronización.

Por esta razón, una optimización efectiva debe intentar mantener la comunicación tan **local** como sea posible.

Cooperative Groups

Las primitivas tradicionales de CUDA permiten sincronizar hilos utilizando mecanismos como `__syncwarp()` o `__syncthreads()`. Sin embargo, estas operaciones están asociadas a unidades específicas de ejecución, como un *warp* o un bloque completo.

Cooperative Groups proporciona una abstracción que permite definir explícitamente **grupos de hilos que cooperan entre sí** y realizar operaciones colectivas sobre estos grupos.

Conceptualmente, en lugar de considerar únicamente:

$$\text{Thread} \rightarrow \text{Warp} \rightarrow \text{Block}, \quad (3.64)$$

es posible definir:

$$\boxed{\text{Grupo} = \{\text{conjunto de hilos que cooperan}\}} \quad (3.65)$$

De esta forma, el algoritmo puede expresar explícitamente el alcance de la cooperación requerida.

Grupo de hilos

Para utilizar Cooperative Groups se incluye:

```
#include <cooperative_groups.h>
```

```
namespace cg = cooperative_groups;
```

Dentro de un kernel puede obtenerse un grupo que representa al bloque actual:

```
__global__ void kernel(float *data) {
    cg::thread_block block = cg::this_thread_block();

    // ...

    block.sync();

    // ...
}
```

En este caso:

```
block.sync();
```

representa una barrera para los hilos pertenecientes al grupo. Para un `thread_block`, su efecto corresponde conceptualmente a una sincronización del bloque completo. La diferencia importante es que la sincronización se expresa como una operación del **grupo que participa en el algoritmo**:

```
group.sync() (3.66)
```

en lugar de utilizar una primitiva cuyo alcance está implícito.

Partición de un bloque

Un bloque puede dividirse en grupos más pequeños. Por ejemplo, considere un bloque de 256 hilos que se divide en grupos de 32:

Block: 256 threads

 (3.67)

$$\underbrace{[0 \dots 31]}_{G_0} \underbrace{[32 \dots 63]}_{G_1} \underbrace{[64 \dots 95]}_{G_2} \dots \underbrace{[224 \dots 255]}_{G_7}. \quad (3.68)$$

Esto puede implementarse utilizando:

```
cg::thread_block block = cg::this_thread_block();

auto tile = cg::tiled_partition<32>(block);
```

Cada hilo obtiene un objeto que representa el grupo al cual pertenece. El identificador del hilo dentro del grupo puede obtenerse mediante:

```
tile.thread_rank();
```

y el número de participantes mediante:

```
tile.size();
```

Por ejemplo:

```
int local_id = tile.thread_rank();
int group_size = tile.size();
```

Esto permite escribir algoritmos en términos del grupo que coopera, en lugar de depender directamente de índices globales o del tamaño completo del bloque.

Sincronización con alcance reducido

Suponga que un bloque contiene 256 hilos, pero una etapa del algoritmo requiere cooperación únicamente entre grupos de 32 hilos. Sincronizar todo el bloque:

```
__syncthreads();
```

impone una barrera sobre los 256 hilos.

Con Cooperative Groups puede expresarse:

```
auto tile = cg::tiled_partition<32>(
    cg::this_thread_block());

tile.sync();
```

de manera que la cooperación se expresa con el alcance requerido por el algoritmo.

El principio de optimización continúa siendo:

Consideración

Utilizar el menor grupo que satisfaga la dependencia

Operaciones colectivas

Cooperative Groups no solamente permite sincronizar. También proporciona operaciones colectivas entre los participantes de determinados grupos. Por ejemplo, es posible intercambiar valores entre hilos mediante operaciones similares a las instrucciones *shuffle*:

```
value = tile.shfl_down(value, offset);
```

Esto permite implementar una reducción:

```
for (int offset = tile.size() / 2;
     offset > 0;
     offset /= 2) {

    value += tile.shfl_down(value, offset);
}
```

Para un grupo de 32 hilos:

$$32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1. \quad (3.69)$$

Cada etapa intercambia información entre los participantes hasta producir un resultado parcial.

La diferencia conceptual con una implementación basada directamente en primitivas de *warp* es que el algoritmo está expresado en términos del **grupo cooperativo**.

Funciones que reciben grupos

Una ventaja importante de Cooperative Groups es que las funciones pueden recibir explícitamente el grupo de hilos que participa en una operación. Por ejemplo:

```
template <typename Group>
__device__ float reduce(Group group, float value)
{
    for (int offset = group.size() / 2;
         offset > 0;
         offset /= 2) {

        value += group.shfl_down(value, offset);
    }

    return value;
}
```

El kernel puede posteriormente utilizar:

```
cg::thread_block block = cg::this_thread_block();
auto tile = cg::tiled_partition<32>(block);

float result = reduce(tile, value);
```

Esto permite separar la lógica de cooperación de la organización específica del kernel. Además, hace explícito que la función:

`reduce(group, value)` (3.70)

es una operación colectiva en la cual deben participar los hilos correspondientes al grupo.

Sincronización a nivel de grid

En un kernel convencional, los bloques deben considerarse independientes. No puede asumirse que todos los bloques estén ejecutándose simultáneamente ni utilizar una barrera convencional para sincronizarlos.

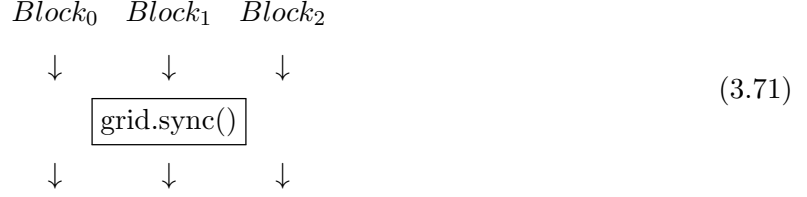
Cooperative Groups también permite, bajo condiciones específicas, crear un grupo que representa el *grid*:

```
cg::grid_group grid = cg::this_grid();
```

y realizar:

```
grid.sync();
```

Conceptualmente:



Esto permite implementar fases globales dentro de un mismo kernel:

$$\text{Fase A} \rightarrow \text{grid.sync()} \rightarrow \text{Fase B.} \tag{3.72}$$

Sin embargo, la sincronización de todo el grid requiere un **lanzamiento cooperativo** y que el dispositivo soporte esta funcionalidad. Además, deben poder cumplirse las restricciones de residencia requeridas para garantizar la cooperación entre los bloques.

Por esta razón, una barrera global debe utilizarse únicamente cuando la estructura del algoritmo realmente la requiera.

Kernel convencional frente a kernel cooperativo

Considere un algoritmo formado por dos fases con una dependencia global.

Una solución convencional utiliza dos kernels:

$$K_A \rightarrow \boxed{\text{fin del kernel}} \rightarrow K_B. \tag{3.73}$$

La frontera entre kernels proporciona un punto natural para separar ambas fases. Con un lanzamiento cooperativo puede ser posible expresar:

$$\boxed{\text{Fase A} \rightarrow \text{grid.sync()} \rightarrow \text{Fase B}} \tag{3.74}$$

dentro del mismo kernel.

Esto puede evitar ciertos costes asociados con dividir el algoritmo en múltiples kernels y permite mantener estado intermedio dentro de una ejecución más persistente.

No obstante, fusionar las fases de esta manera puede incrementar el consumo de registros, memoria compartida y otros recursos. Además, la necesidad de mantener bloques cooperativos residentes puede limitar el tamaño del grid.

Por tanto:

$$\boxed{\text{menos fronteras entre kernels} \quad \longleftrightarrow \quad \text{mayores restricciones de ejecución}} \tag{3.75}$$

Jerarquía de cooperación

Cooperative Groups permite observar la ejecución paralela como una jerarquía de grupos:

$$\boxed{\text{Thread} \subset \text{Tile} \subseteq \text{Block} \subseteq \text{Grid}} \quad (3.76)$$

Cada nivel permite diferentes formas de comunicación y sincronización.

Grupo	Participantes	Uso típico
Thread	1 hilo	Trabajo independiente
Tile	Subconjunto del bloque	Reducciones y comunicación local
Thread Block	Hilos del bloque	Shared memory y tiling
Grid	Bloques del kernel	Cooperación global

Esta jerarquía permite seleccionar el alcance de cooperación de acuerdo con las dependencias reales del algoritmo.

Cooperative Groups como herramienta de optimización

Cooperative Groups no debe interpretarse como un mecanismo que automáticamente acelera un kernel. Su principal ventaja es permitir expresar explícitamente la estructura de cooperación del algoritmo.

Desde el punto de vista de optimización, esto facilita:

- limitar la sincronización a los hilos que realmente cooperan;
- implementar operaciones colectivas sobre subconjuntos de hilos;
- construir funciones reutilizables que reciben explícitamente un grupo;
- organizar algoritmos jerárquicos;
- implementar determinadas fases globales dentro de kernels cooperativos;
- evitar algunas sincronizaciones innecesariamente amplias.

La estrategia general consiste nuevamente en mantener la comunicación y la sincronización tan locales como sea posible:

$$\boxed{\text{Thread} \rightarrow \text{Tile} \rightarrow \text{Block} \rightarrow \text{Grid}} \quad (3.77)$$

y avanzar hacia un grupo de mayor alcance únicamente cuando las dependencias del algoritmo lo requieran.

Unidades y operaciones especializadas

Hasta este punto, las optimizaciones se han concentrado principalmente en incrementar el paralelismo, reducir el movimiento de datos y organizar eficientemente la comunicación entre hilos. Sin embargo, el rendimiento también depende de **qué operaciones ejecuta cada hilo** y de las unidades funcionales disponibles para realizarlas.

Las GPU modernas poseen unidades especializadas para diferentes clases de operaciones. De manera conceptual, un multiprocesador puede disponer de recursos para:

- operaciones enteras;
- operaciones de punto flotante;
- operaciones de precisión reducida;
- funciones matemáticas especiales;
- operaciones matriciales.

Por tanto, dos expresiones matemáticamente equivalentes pueden producir secuencias de instrucciones diferentes y utilizar recursos distintos del hardware. Desde el punto de vista de optimización, resulta útil considerar tres mecanismos:

1. uso de *intrinsic*s;
2. utilización de tipos de datos especializados o de precisión reducida;
3. utilización de operaciones aproximadas cuando la aplicación tolera un error numérico.

Intrinsics

Un *intrinsic* es una función proporcionada por el compilador que permite expresar directamente una operación soportada o especialmente optimizada para la arquitectura.

A diferencia de una función convencional, un *intrinsic* puede ser reconocido por el compilador y transformado directamente en una o pocas instrucciones de máquina.

Por ejemplo, una operación común es el cálculo:

$$d = ab + c. \tag{3.78}$$

Una implementación puede escribirse como:

```
float d = a * b + c;
```

mientras que CUDA permite expresar explícitamente una operación *fused multiply-add*:

```
float d = __fmaf_rn(a, b, c);
```

Una operación FMA calcula conceptualmente:

$$d = ab + c \quad (3.79)$$

como una operación fusionada, evitando un redondeo intermedio entre la multiplicación y la suma.

Además de posibles ventajas de rendimiento, esto puede modificar ligeramente el resultado numérico con respecto a ejecutar una multiplicación y una suma independientes.

Los *intrinsic*s también permiten especificar determinados comportamientos aritméticos, como modos de redondeo. Por ejemplo:

```
__fadd_rn(a,b)
__fadd_rz(a,b)

__fmul_rn(a,b)
__fmul_rz(a,b)
```

donde los sufijos indican diferentes políticas de redondeo.

Por tanto, los *intrinsic*s constituyen una forma de proporcionar al programador mayor control sobre la operación que se desea ejecutar.

Funciones matemáticas especiales

Operaciones como:

$$\sin(x), \quad \cos(x), \quad 2^x, \quad \log_2(x), \quad \frac{1}{x}, \quad \sqrt{x} \quad (3.80)$$

son considerablemente más complejas que una suma o una multiplicación.

Las GPU pueden incluir recursos especializados para acelerar determinadas funciones trascendentales y operaciones matemáticas. En arquitecturas NVIDIA, estas capacidades se asocian comúnmente con las *Special Function Units* (SFU).

CUDA proporciona versiones *intrinsic* de diferentes funciones. Algunos ejemplos son:

```
__sinf(x)
__cosf(x)
__expf(x)
__logf(x)
__powf(x,y)
```

Estas funciones pueden utilizar implementaciones especialmente optimizadas para la arquitectura. Conceptualmente, en lugar de implementar una función compleja mediante una larga secuencia de operaciones:

$$f(x) \rightarrow \{op_0, op_1, \dots, op_N\}, \quad (3.81)$$

puede aprovecharse una ruta especializada:

$$f(x) \rightarrow \boxed{\text{unidad/implementación especializada}}. \quad (3.82)$$

Precisión y rendimiento

Las funciones matemáticas no necesariamente poseen una única implementación.

Considere:

`sinf(x)`

y:

`__sinf(x)`

Ambas calculan una aproximación al seno, pero pueden presentar diferentes compromisos entre precisión, rango de entrada, coste y rendimiento. De manera conceptual:

$$\boxed{\text{mayor precisión} \longleftrightarrow \text{menor coste computacional}} \quad (3.83)$$

El uso de una versión más rápida solamente resulta apropiado cuando el error introducido es aceptable para la aplicación. Por ejemplo, los requisitos numéricos pueden ser muy diferentes entre:

- una simulación científica;
- un filtro de procesamiento de imágenes;
- una transformación gráfica;
- un algoritmo de percepción;
- la inferencia de una red neuronal.

La optimización debe considerar, por tanto, el **error permitido por el problema** y no únicamente la velocidad de una operación.

Fast Math

CUDA permite solicitar al compilador el uso de determinadas operaciones matemáticas de menor coste mediante:

```
--use_fast_math
```

Esta opción modifica diferentes operaciones de punto flotante para favorecer implementaciones de mayor rendimiento, potencialmente reduciendo precisión o modificando determinados comportamientos numéricos.

Por ejemplo, algunas operaciones pueden utilizar versiones intrínsecas equivalentes a:

$$\text{sinf}(x) \rightarrow _ \text{sinf}(x). \quad (3.84)$$

El objetivo puede representarse como:

$$\boxed{\text{precisión estricta} \rightarrow \text{precisión suficiente} \rightarrow \text{mayor rendimiento}} \quad (3.85)$$

Sin embargo, `--use_fast_math` afecta múltiples operaciones del programa. Por esta razón, debe utilizarse después de analizar el efecto del cambio sobre la precisión final de la aplicación.

Una alternativa consiste en utilizar explícitamente versiones rápidas únicamente en las regiones donde el error ha sido caracterizado.

Tipos de precisión reducida

Otra estrategia consiste en modificar el tipo de dato utilizado para representar los operandos. Tradicionalmente, una aplicación puede utilizar:

```
double
float
```

correspondientes, respectivamente, a representaciones de 64 y 32 bits. Sin embargo, las GPU modernas soportan formatos adicionales de menor precisión.

Entre ellos se encuentran:

Tipo	Bits	Uso típico
FP64	64	Cómputo científico
FP32	32	Cómputo general
FP16	16	IA y procesamiento numérico
BF16	16	Entrenamiento e inferencia de IA
FP8	8	IA de precisión reducida
INT8	8	Inferencia y operaciones enteras
INT4	4	Inferencia cuantizada

Reducir el número de bits puede proporcionar beneficios en diferentes partes del sistema. En primer lugar, disminuye el almacenamiento:

$$N \times 32 \text{ bits} \rightarrow N \times 16 \text{ bits.} \quad (3.86)$$

En segundo lugar, disminuye el tráfico de memoria. Si el mismo número de elementos puede representarse con la mitad de bits, se reduce la cantidad de información que debe transferirse. Por ejemplo:

$$10^6 \text{ valores FP32} \approx 4 \text{ MB,} \quad (3.87)$$

mientras que:

$$10^6 \text{ valores FP16} \approx 2 \text{ MB.} \quad (3.88)$$

Por tanto, la precisión reducida puede beneficiar incluso a algoritmos limitados por memoria.

Tipos especiales en CUDA

CUDA proporciona tipos específicos para trabajar con representaciones de precisión reducida. Por ejemplo, para FP16:

```
#include <cuda_fp16.h>

__half a;
__half b;
```

También existen tipos que permiten agrupar múltiples valores:

```
__half2
```

que representa dos valores FP16. Conceptualmente:

$$\boxed{\underbrace{FP16}_{16} + \underbrace{FP16}_{16}} = \boxed{\underbrace{half2}_{32 \text{ bits}}}. \quad (3.89)$$

Esto permite que determinadas instrucciones operen sobre varios valores de precisión reducida simultáneamente. Por ejemplo:

```
__half2 a;
__half2 b;

__half2 c = __hadd2(a, b);
```

realiza conceptualmente:

$$(c_0, c_1) = (a_0 + b_0, a_1 + b_1). \quad (3.90)$$

Este tipo de operaciones introduce una forma adicional de paralelismo dentro de cada hilo.

Paralelismo a nivel de instrucción

Hasta ahora se ha estudiado principalmente el paralelismo generado mediante múltiples hilos:

$$\boxed{\text{muchos hilos} \rightarrow \text{muchas operaciones}} \quad (3.91)$$

Sin embargo, tipos empaquetados como `half2` permiten procesar múltiples elementos mediante una operación:

$$\boxed{\text{un hilo} \rightarrow \text{una instrucción} \rightarrow \text{múltiples datos}} \quad (3.92)$$

Por tanto, el rendimiento puede aprovechar paralelismo en diferentes niveles:

$$\boxed{\text{Threads} \times \text{Warps} \times \text{Instruction-level parallelism}} \quad (3.93)$$

Operaciones matriciales especializadas

Las GPU orientadas a cargas de inteligencia artificial incorporan además unidades especializadas para operaciones matriciales.

Una operación fundamental es:

$$D = AB + C. \quad (3.94)$$

En lugar de implementar cada producto escalar exclusivamente mediante instrucciones escalares convencionales, unidades matriciales especializadas pueden ejecutar múltiples operaciones multiply-accumulate sobre pequeños bloques de matrices. Conceptualmente:

$$\boxed{\text{FP32/FP16 ALUs}} \quad \text{vs.} \quad \boxed{\text{Matrix/Tensor Units}}. \quad (3.95)$$

Estas unidades son particularmente importantes en cargas como:

- multiplicación de matrices;
- convoluciones;
- redes neuronales;
- mecanismos de atención;
- procesamiento de modelos de lenguaje.

Su utilización suele requerir estructuras de datos, dimensiones y tipos compatibles con las operaciones soportadas por el hardware.

Aritmética aproximada

Una optimización más general consiste en preguntarse si es necesario calcular exactamente la operación originalmente especificada.

Suponga que una función requiere:

$$y = f(x). \quad (3.96)$$

Una implementación aproximada puede utilizar:

$$\hat{y} = \hat{f}(x), \quad (3.97)$$

donde:

$$\hat{f}(x) \approx f(x) \quad (3.98)$$

pero posee un coste computacional menor.

El error puede caracterizarse mediante:

$$e(x) = |f(x) - \hat{f}(x)|. \quad (3.99)$$

La decisión deja entonces de ser únicamente:

$$\text{¿cuál implementación es más rápida?} \quad (3.100)$$

y se convierte en:

$$\boxed{\text{¿cuál es la implementación de menor coste que mantiene un error aceptable?}} \quad (3.101)$$

Aproximaciones matemáticas

Una función compleja puede sustituirse por diferentes aproximaciones. Por ejemplo:

$$e^x \tag{3.102}$$

puede aproximarse mediante un polinomio:

$$e^x \approx 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!}. \tag{3.103}$$

También pueden utilizarse:

- tablas de búsqueda (*LUT*);
- interpolación;
- aproximaciones polinomiales;
- aproximaciones por segmentos;
- reducción de precisión;
- algoritmos iterativos con menor número de iteraciones.

Cada alternativa presenta un compromiso diferente entre:

error	↔	latencia	↔	throughput	↔	movimiento de datos
-------	---	----------	---	------------	---	---------------------

(3.104)

Optimización basada en precisión suficiente

El objetivo de una implementación eficiente no debe ser necesariamente utilizar la menor precisión disponible. La reducción de precisión solamente es útil si el resultado continúa satisfaciendo los requisitos del sistema.

Una metodología apropiada consiste en:

1. establecer una implementación de referencia;
2. determinar la precisión requerida por la aplicación;
3. identificar las operaciones computacionalmente costosas;
4. evaluar tipos de menor precisión o implementaciones aproximadas;
5. medir el error introducido;
6. medir el cambio en rendimiento;
7. seleccionar el compromiso apropiado.

Este principio puede resumirse como:

$$\boxed{\text{No utilizar mayor precisión que la necesaria}} \quad (3.105)$$

La reducción de precisión puede producir simultáneamente menos almacenamiento, menor tráfico de memoria y acceso a unidades funcionales de mayor throughput. Sin embargo, estos beneficios dependen de que el hardware posea soporte eficiente para el tipo seleccionado y de que las conversiones entre formatos no introduzcan un coste que elimine las ganancias obtenidas.